

Practica 4: Clasificadores

Dra. María Elena Martínez Perez
M. en C. Miguel Ángel Veloz Lucas

29 de abril de 2026

1 Reglas generales para el desarrollo de las Prácticas de Laboratorio

- Deberás respetar la estructura general de este documento, i.e, entregar tu práctica con las secciones: objetivos, introducción, desarrollo, código, conclusiones y referencias.
- El desarrollo de la práctica deberá ser auténtico. Aquellos equipos que presenten los mismos cálculos, código fuente, etcétera, serán sancionados.
- El día de entrega establecido deberá ser respetado por todos los equipos. La hora límite de entrega es la hora de clase y no se reciben trabajos que sean terminados durante la hora de clase.
- Deberás entregar el documento en el classroom antes de la fecha límite.

2 Objetivos

En esta práctica avanzaremos en el uso de cuatro clasificadores supervisados sobre distintos conjuntos de datos. Cada ejercicio incluye:

- Preprocesamiento de datos.
- Reducción de dimensionalidad PCA y visualización en 2D/3D.
- Búsqueda e interpretación de hiperparámetros.
- Análisis de resultados.

3 Introducción

3.1 Tipos de Clasificadores para Reconocimiento de Patrones

El reconocimiento de patrones es una disciplina fundamental en inteligencia artificial y aprendizaje automático, cuyo objetivo es identificar regularidades o

estructuras en datos para asignarles categorías. Los clasificadores son algoritmos que implementan esta tarea, y su elección depende de factores como la naturaleza de los datos, la complejidad del problema y la necesidad de interpretabilidad. A continuación, se describen los principales tipos de clasificadores y sus características.

3.2 Clasificadores No Lineales

Cuando los datos presentan relaciones complejas, se emplean modelos no lineales. Los árboles de decisión son un ejemplo: dividen recursivamente el espacio de características mediante reglas jerárquicas, lo que los hace interpretables pero propensos al sobreajuste. Otro enfoque es el algoritmo k-vecinos más cercanos (kNN), que clasifica instancias según la mayoría de clases entre sus vecinos más cercanos. Aunque es flexible y no requiere entrenamiento explícito, su costo computacional crece con el tamaño de los datos. Estos métodos son útiles en reconocimiento de escritura manual o sistemas de recomendación basados en similitud.

3.3 Clasificadores Probabilísticos

Basados en teoría de probabilidad, estos modelos estiman distribuciones para predecir clases. El clasificador Naive Bayes es el más conocido: asume independencia entre características y aplica el teorema de Bayes para calcular probabilidades. Aunque simplista, es eficaz en datos categóricos o textuales, como en análisis de sentimientos o detección de spam. Las redes Bayesianas, por otro lado, modelan dependencias entre variables, ofreciendo mayor flexibilidad a costa de complejidad.

3.4 Máquinas de Soporte Vectorial (SVM)

Las SVM buscan hiperplanos óptimos para separar clases, incluso en espacios de alta dimensión. En su versión lineal, maximizan el margen entre clases, mientras que con *kernels* transforman los datos a espacios donde la separación no lineal es posible. Son eficaces en reconocimiento facial o clasificación de imágenes médicas, pero requieren ajuste de parámetros y normalización de datos.

3.5 Redes Neuronales

Estos modelos, inspirados en el cerebro humano, destacan por su capacidad para aprender patrones complejos. Los perceptrones multicapa (MLP) son redes *feed-forward* con capas ocultas no lineales, mientras que las redes convolucionales (CNN) se especializan en datos *grid-like* (como imágenes). Las redes recurrentes (RNN) manejan secuencias (texto, series temporales). Aunque son extremadamente poderosas (ej. en visión por computadora o traducción automática), requieren grandes volúmenes de datos, recursos computacionales y carecen de interpretabilidad.

4 Desarrollo

Ejercicio: Aplicación y comparación de clasificadores.

Selecciona tres conjuntos de datos para entrenar distintos modelos de clasificación variando la configuración de sus hiperparámetros.

Conjunto de datos propuesto:

- Clasificación Multimodelo para Cáncer de Mama (*Breast Cancer Wisconsin Diagnostic*): Conjunto de datos biomédicos que contiene características numéricas derivadas de imágenes digitales de aspiraciones con aguja fina (FNA) de masas mamarias, utilizadas para clasificar tumores como benignos o malignos.
- Detección Temprana de Parkinson, voz y biomarcadores acústicos (*Breast Parkinsons Telemonitoring*): Es un conjunto de datos diseñado para apoyar la investigación en el diagnóstico del enfermedad de Parkinson. Contiene medidas biomédicas de voz obtenidas de personas sanas y pacientes con la enfermedad, con el objetivo de diferenciar entre ambos grupos mediante técnicas de aprendizaje automático.
- Clasificación de Cardiopatía (*Heart Disease*): Este conjunto de datos recopila información de pacientes para determinar la probabilidad de enfermedad cardíaca, codificada en una variable objetivo que indica la presencia o ausencia de la condición. Entre las variables predictoras se incluyen edad, sexo, presión arterial en reposo, nivel de colesterol, frecuencia cardíaca máxima y resultados de pruebas electrocardiográficas.
- Verificador de síntomas de COVID-19 (*COVID Symptom Classification Dataset*): Los datos identifican si una persona padece o no la enfermedad del coronavirus, basándose en algunos síntomas estándar predefinidos. Dichos síntomas se basan en las directrices de la Organización Mundial de la Salud (OMS) y del Ministerio de Salud y Bienestar Familiar de la India.

1. Carga y análisis de datos:

- (a) Cargar el conjunto de datos.
- (b) Visualización de la distribución de clases.
- (c) Análisis de valores faltantes (porcentaje de valores faltantes por columna).

2. Preprocesamiento y limpieza:

- (a) Separación de tipos de variables (Identificamos columnas numéricas y categóricas).
- (b) Verificar y aplicar limpieza de datos (faltantes, *outliers*, caracteres inválidos, etc.).

- (c) Estandarización de características (escaladores).
 - (d) Separación en 70% entrenamiento y 30% prueba.
 - (e) Manejo del desbalance de clases sobre el conjunto de entrenamiento (por ejemplo SMOTE, ADASYN o Sobremuestreo aleatorio).
 - (f) Visualización de la distribución.
3. **Reducción de dimensionalidad por análisis de componentes principales (PCA):**
- (a) Aplicar PCA a los datos balanceados.
 - (b) Cálculo de la varianza acumulada.
 - (c) Encontrar el número de componentes para el 95% de varianza.
 - (d) Gráfico del número de componentes principales y varianza acumulada.
4. **Definición de modelos, esquema de trabajo y búsqueda en cuadrícula (grid search):**
- (a) Desarrollar un esquema de funciones (*pipelines*) que integren el preprocesamiento y el clasificador.
 - (b) La búsqueda en cuadrícula nos permitirá encontrar los mejores hiperparámetros. Este método consiste en especificar una lista de posibles valores para cada hiperparámetro y luego entrenar el modelo para cada combinación de estos valores. Se evalúa el rendimiento de cada modelo y se selecciona la combinación de hiperparámetros que da el mejor rendimiento. Este método es exhaustivo y puede resultar computacionalmente costoso, especialmente cuando se trata con un gran número de hiperparámetros o un gran número de datos.
5. **Configuración de modelos y cuadro de hiperparámetros**
- (a) Modelo de KNN:
 - vecinos (1,3,5,7,9).
 - pesos (*weights*: ['uniform', 'distance']).
 - distancia (*metric*: ['euclidean', 'manhattan']).
 - (b) Modelo de árbol de decisión
 - profundidad ([3, 5, 7, None]).
 - número mínimo de muestras por división (*samples split*: [2, 5, 10]).
 - criterio de división (*criterion*: ['gini', 'entropy']).
 - (c) Modelo de SVM
 - kernel (['linear', 'rbf']).
 - Regularización C (0.1, 1, 10).

- Gamma (`['scale', 'auto', 0.01, 0.1]`).

6. Entrenamiento, evaluación y comparación de resultados:

- Mejores métricas de rendimiento sobre conjunto de prueba por conjunto de datos.
- Matriz de confusión de los modelos entrenados con las mejores métricas.
- Mostrar las fronteras de decisión superponiendo datos de entrenamiento y prueba.
- Mostrar el árbol entrenado con los mejores parámetros.
- Comparar exactitud entre modelos.
- ¿Qué hiperparámetro fue más relevante por modelo para cada conjunto de datos?

5 Código

En esta sección deberás presentar el código fuente del programa que hayas escrito.

6 Conclusiones

En esta sección deberás escribir las conclusiones a las que hayas llegado una vez terminada la práctica.

7 Referencias

- Tolstikhin, Ilya O., et al. *Mlp-mixer: An all-mlp architecture for vision*. Advances in neural information processing systems 34 (2021): 24261-24272.
- Wu, Baoyuan, et al. *Defenses in adversarial machine learning: A survey*. arXiv preprint arXiv:2312.08890 (2023).
- Liu, Xiao, et al. *Self-supervised learning: Generative or contrastive*. IEEE transactions on knowledge and data engineering 35.1 (2021): 857-876.
- Goodfellow, Ian. *Deep learning*. (2016).
- Murphy, Kevin P. *Machine learning: a probabilistic perspective*. MIT press, 2012.
- Wolberg, W., Mangasarian, O., Street, N., Street, W. (1993). Breast Cancer Wisconsin (Diagnostic) [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5DW2B>.

- Tsanas, A. Little, M. (2009). Parkinsons Telemonitoring [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5ZS3N>.
- Janosi, A., Steinbrunn, W., Pfisterer, M., Detrano, R. (1989). Heart Disease [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C52P4X>.
- COVID Symptom Classification [Dataset]. (2021) Kaggle Repository <https://www.kaggle.com/datasets/iamhungundji/covid19-symptoms-checker>.